

Team members: Ryan Salute, Winnie Trinh, Karina Lee

Design decisions & implementation approach

Matrix multiplication using MPI

The MPI program multiplies two 1000×1000 matrices. Rank 0 initializes matrices A and B using a fixed random seed so that each run uses the same input values.

The work is divided by rows of matrix A. Rank 0 broadcasts the full matrix B to every MPI process. It then distributes contiguous row blocks of A using MPI_Scatterv. Each process independently calculates the corresponding rows of result matrix C. Finally, the partial result rows are returned to rank 0 using MPI_Gatherv.

MPI_Scatterv and MPI_Gatherv were selected instead of fixed-size scatter/gather calls so the program can support process counts that do not evenly divide the matrix size. The execution time reported is the time for the distributed matrix multiplication, including communication and local computation. The reported time uses the slowest MPI rank because the full program cannot finish until all ranks complete.

Sequential and goroutine string search

The string-search program searches a 1 MiB text file named data.txt for a user-provided search string. The search string is passed through the -search command-line flag. The sequential version examines every possible starting position in the file and counts every matching occurrence.

The goroutine version divides the text into approximately equal byte ranges. Each goroutine searches one range and stores its local occurrence count. The main goroutine waits for all workers using a sync.WaitGroup and adds the local counts. Each worker can read a small number of bytes past the end of its assigned range. This overlap ensures that a match crossing a chunk boundary is not missed, while only the worker that owns the match's starting position counts it.

RPC string search

The RPC implementation uses Go's net/rpc library. The client reads data.txt, divides the file into chunks, and sends one chunk to each RPC worker. Every worker runs a SearchWorker.Count RPC method and returns its local occurrence count. The client combines the returned counts into the final result.

The RPC implementation uses persistent client connections across timing runs. This means the measured time includes request encoding, text transfer, worker computation, reply transfer, and decoding, but excludes one-time connection setup. The same boundary-overlap strategy used by the goroutine version prevents missed matches at chunk boundaries.

data.txt contains 9,000 occurrences of the search string needle.

Explanation of correctness

Correctness was verified by running the sequential, goroutine, and RPC string-search implementations with the same input file and search string:

-search needle

All implementations returned 9,000 occurrences. The result remained the same when the goroutine and RPC worker count changed from 1 to 2 to 4. This confirms that splitting the input across workers did not lose or duplicate occurrences.

MPI matrix multiplication correctness was verified by having rank 0 calculate a sequential reference matrix after the distributed multiplication completes. The program compares every value in the gathered distributed result matrix against the corresponding value in the sequential reference matrix. Every tested MPI configuration printed Verification: PASS.

Instructions to compile & run each program:

Lab 2 commands

Run commands from `Lab2`. `data.txt` is exactly 1 MiB; regenerate it with `go run make\_text.go` if needed.

Sequential and goroutine search

```
go run wordsearch_seq.go -file data.txt -search needle -runs 20
go run wordsearch_par.go -file data.txt -search needle -workers 1 -runs 20
go run wordsearch_par.go -file data.txt -search needle -workers 2 -runs 20
go run wordsearch_par.go -file data.txt -search needle -workers 4 -runs 20
```

RPC search

Start one worker per terminal:

```
go run rpc_worker.go -listen :9001
go run rpc_worker.go -listen :9002
go run rpc_worker.go -listen :9003
go run rpc_worker.go -listen :9004
```

Run the 1-, 2-, and 4-worker tests respectively:

```
go run wordsearch_rpc.go -file data.txt -search needle -addrs localhost:9001 -runs 20
go run wordsearch_rpc.go -file data.txt -search needle -addrs localhost:9001,localhost:9002 -runs 20
go run wordsearch_rpc.go -file data.txt -search needle -addrs
localhost:9001,localhost:9002,localhost:9003,localhost:9004 -runs 20
```

MPI matrix multiplication

```
mpicc -O3 -Wall -Wextra matrix_mpi.c -o matrix_mpi
mpirun -np 1 ./matrix_mpi
mpirun -np 2 ./matrix_mpi
mpirun -np 4 ./matrix_mpi
```

Design decisions & implementation approach:

Data.txt has 9000 occurrences?

Explanation of correctness:

Timing & Performance Tables:

RPC workers	Occurrences	Average search time (10 runs)
1	9000	4.788 ms
2	9000	2.995 ms
4	9000	4.959 ms

Version	Workers	Occurrences	Average time	Speedup vs. sequential
Sequential	1	9000	0.773 ms	1.000×
Goroutines	1	9000	1.369 ms	0.565×
Goroutines	2	9000	0.879 ms	0.879×
Goroutines	4	9000	0.540 ms	1.431×
RPC	1	9000	4.788 ms	0.161×

RPC	2	9000	2.995 ms	0.258×
RPC	4	9000	4.959 ms	0.156×

MPI processes	Time (s)	Verification	Speedup	Efficiency
1	2.180457	PASS	1.000	100.0%
2	1.082717	PASS	2.014	100.7%
4	0.555987	PASS	3.922	98.0%
8	0.619098	PASS	3.522	44.0%
12	0.451176	PASS	4.833	40.3%
13	0.424170	PASS	5.141	39.5%
14	0.393441	PASS	5.542	39.6%

Speedup calculations & performance analysis:

Speedup was calculated as 0.773 sequential time divided by the other versions' average time.

Conclusion: optimal number of MPI processes

Among the tested MPI configurations, 14 processes produced the best execution time:

MPI processes	Time (s)
14	0.393441

Therefore, 14 MPI processes was the optimal tested configuration for this matrix size and test system. It achieved a speedup of 5.542× compared with one MPI process.

Contributions:

Karina Lee: coding, questions, debugging

Ryan Salute: coding, questions, debugging

Winnie Trinh: coding, questions, debugging